FACULTAD DE CIENCIAS EXACTAS
UNIVERSIDAD NACIONAL DEL CENTRO DE LA PROVINCIA DE BUENOS AIRES

BASE DE DATOS 2

Consultas de Datos



INGENIERÍA DE SISTEMAS

Lenguaje SQL

Algunas funciones del estandar SQL:

DDL
- ✓ Definición de datos
  - • Creación de objetos (CREATE)
  - • Modificación de objetos definidos (ALTER)
  - • Eliminación de objetos definidos (DROP)

Los objetos pueden ser:
- *tablas,*
- *vistas,*
- *índices,*
- *dominios,*
- *etc.*

DML
- ✓ Actualización de los datos
  - • Inserción (INSERT)
  - • Actualización (UPDATE)
  - • Eliminación (DELETE)
- ✓ Consulta de datos
  - • Selección (SELECT)

Lenguaje de Consulta SQL

- ✓ Para la recuperación de los datos a partir de tablas cargadas en la base de datos se utiliza la sentencia **SELECT**.
- ✓ El formato más simple y básico es:

```
SELECT * | { [DISTINCT] columna | expresion [alias],...}
FROM  <lista tablas>
```

- ✓ En una consulta se especifica qué información se requiere, sin especificar cómo obtenerla, no le especificamos métodos de acceso a los datos.
- ✓ **SELECT** identifica las columnas a recuperar – **EL QUE**
- ✓ **FROM** identifica la tabla - **DE DONDE obtener los datos**
- ✓ El resultado de una consulta es una *tabla* (si es un número, se considera como una tabla con una fila y una columna).

Cómo funcionan todos los DBMS ?

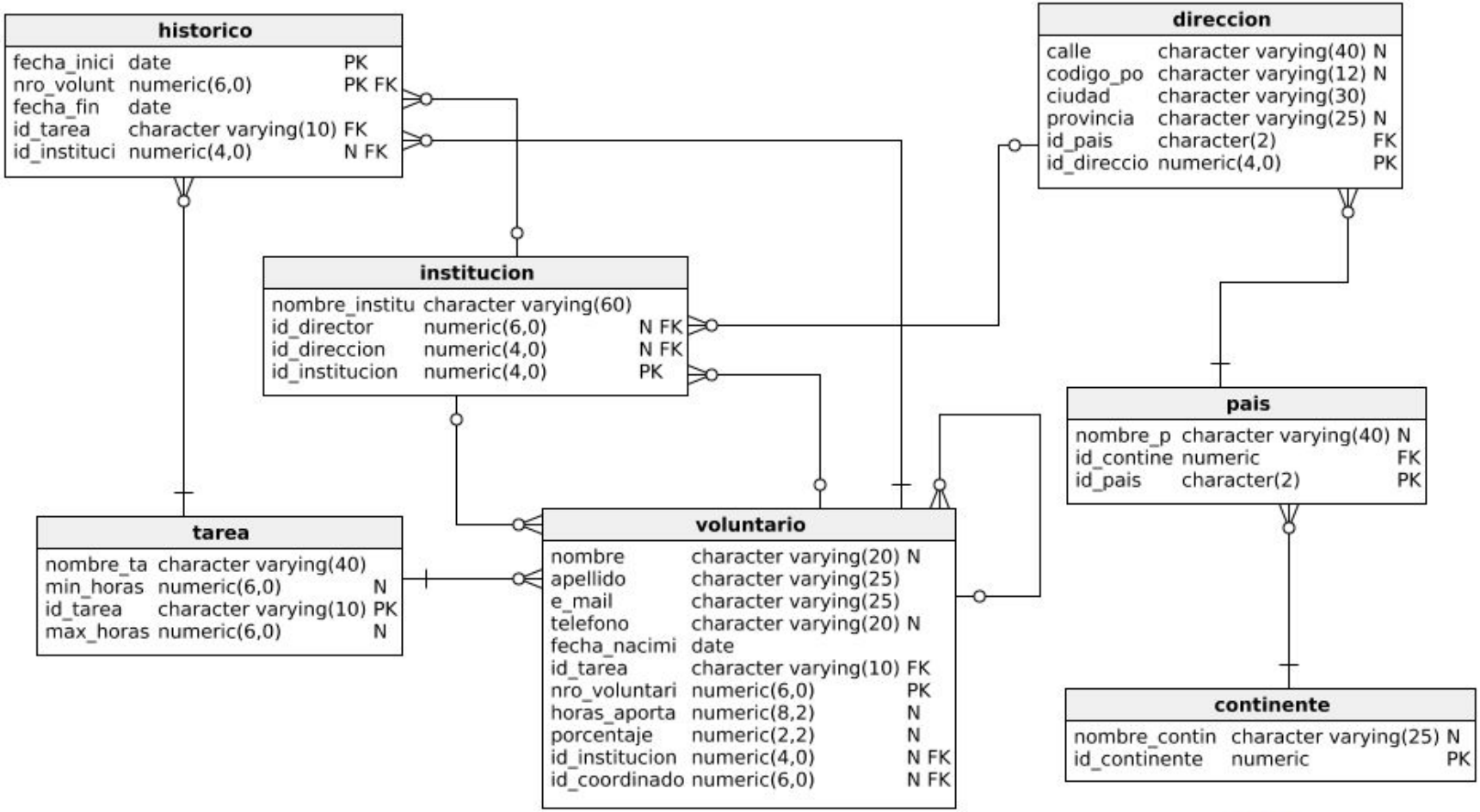
Todos, absolutamente todos los DBMS para resolver una consulta siguen el siguiente esquema.

1) Origen de Datos, el DBMS recupera todas las filas y columnas con las que va a trabajar (FROM)

2) Filtrado del Origen de Datos (WHERE)

3) Proyección (SELECT)

Esquema Ejemplo DERE BD Voluntarios



Vertabelo

Consultas SQL básicas

- ✓ Selección de todas las columnas. Ejemplo: selección de los datos completos de las instituciones.

```
SELECT *
FROM institucion;
```

nombre_institucion	id_director	id_direccion	id_institucion
CASA DE LA PROVIDENCIA	200	1700	10
CORPORACION URRACAS DE EMAUS	201	1800	20
FUNDACION CIVITAS	114	1700	30
FUNDACION LAS ROSAS DE AYUDA FRATERNA	203	2400	40
FUNDACION HOGAR DE CRISTO	121	1500	50
FUNDACION MI CASA	103	1400	60
CORPORACION SOLIDARIDAD Y DESARROLLO	204	2700	70
FUNDACION REGAZO	145	2500	80
FUNDACION ALERTA BOSQUES	100	1700	90
BOSQUEDUCA	108	1700	100
COMITE NACIONAL PRO DEFENSA DE LA FLORA Y LA FAUNA	205	1700	110
CONSEJO ECOLOGICO COMUNAL	*NULL*	1700	120
CORPORACION AMBIENTAL	*NULL*	1700	130
FUNDACION VIDA RURAL	*NULL*	1700	140
CENTRO DE AYUDA MAPUCHE	*NULL*	1700	150
SIERRAS PROTEGIDAS	*NULL*	1700	160
CENTRO DE EDUCACION AMBIENTAL	*NULL*	1700	170
RENACE- RED DE ACCION ECOLOGICA	*NULL*	1700	180
Contracting	*NULL*	1700	190
CONSEJO NACIONAL DE LA JUVENTUD	*NULL*	1700	200
DEFENSA DE LOS DERECHOC DEL NIÑO	*NULL*	1700	210
FUNDACION CHILDREN	*NULL*	1700	220
CORPORACION ANGLICANA	*NULL*	1700	230
CORPORACION EVANGELICA	*NULL*	1700	240
CENTRO ECUMENICO	*NULL*	1700	250

Se usa para especificar la recuperación de todos los datos de la/s tabla/s

Porque se recomienda **evitar** su uso? Y en su lugar seleccionar **SOLO** los campos necesarios...

Ademas:27 fila(s)
Tiempo total de ejecución: 2.589 ms
SQL ejecutada.

Consultas SQL básicas

- ✓ Selección SÓLO de algunas columnas. Ejemplo: *seleccionar el código y el nombre de las instituciones.*

```
SELECT id_institucion, nombre_institucion
FROM  institucion;
```

id_institucion	nombre_institucion
10	CASA DE LA PROVIDENCIA
20	CORPORACION URRACAS DE EMAUS
30	FUNDACION CIVITAS
40	FUNDACION LAS ROSAS DE AYUDA FRATERNA
50	FUNDACION HOGAR DE CRISTO
60	FUNDACION MI CASA
70	CORPORACION SOLIDARIDAD Y DESARROLLO
80	FUNDACION REGAZO
90	FUNDACION ALERTA BOSQUES
100	BOSQUEDUCA
110	COMITE NACIONAL PRO DEFENSA DE LA FLORA Y LA FAUNA
120	CONSEJO ECOLOGICO COMUNAL
130	CORPORACION AMBIENTAL
140	FUNDACION VIDA RURAL
150	CENTRO DE AYUDA MAPUCHE
160	SIERRAS PROTEGIDAS
170	CENTRO DE EDUCACION AMBIENTAL
180	RENACE- RED DE ACCION ECOLOGICA
190	Contracting

Filas Duplicadas

- Por defecto, ante una consulta, se recuperan todas las filas, **incluidas** las filas duplicadas. Ejemplo: *Seleccionar los voluntarios que son coordinadores.*

```
SELECT id_coordinador
FROM  voluntario;
```



id_coordinador
NULL
100
100
102
103
103
103
103
101
108
108
108
108
108
100
114
114
114
114
100
100
100
100
100
120
120
120
120

...

- Para eliminar los valores repetidos se debe usar la cláusula **DISTINCT**. Ejemplo: *Seleccionar los distintos voluntarios que son coordinadores.*

```
SELECT DISTINCT id_coordinador
FROM  voluntario;
```



id_coordinador
NULL
205
122
120
101
103
108
145
100
201
124
114
121
123
102
146
147
148
149

Eliminar los valores repetidos

- La cláusula **DISTINCT** se aplica a todas las columnas de la lista en el **SELECT**. Ejemplo: *Seleccionar los voluntarios coordinadores y las distintas instituciones de los empleados coordinados.*

```
SELECT DISTINCT id_institucion,
                id_coordinador
FROM  voluntario;
```

id_institucion	id_coordinador
90	*NULL*
110	101
100	101
40	101
80	100
110	205
100	108
30	100
50	100
80	147
80	148
60	102
30	114
50	124
80	149
50	121
50	123
80	146
60	103
20	100
20	201
90	100
70	101
80	145
10	101
50	122
NULL	149
50	120

Ejercicios

1. ¿Cuántos registros tiene la tabla tareas?
2. ¿Cuál es el resultado si se listan los códigos de las tareas que están realizando efectivamente los voluntarios? ¿Cuántos registros retorna la consulta?
3. Y si se listan los distintos códigos de las tareas que están realizando efectivamente los voluntarios?

http://dbases.exa.unicen.edu.ar/phppgadmin

Filtrar filas recuperadas

- La cláusula **WHERE** contiene condiciones lógicas que utilizan:
  - operadores de comparación **(<, >, =, <=, >=, <>, !=)**
  - operadores lógicos (**AND, OR, NOT**).
- Las filas recuperadas son aquellas cuyos datos satisfacen todas la/s condición/es lógicas

```
SELECT * | { [DISTINCT] columna | expresion [alias],...}
FROM  <lista tablas>
 [WHERE condicion/es];
```

Filtrar filas recuperadas

Ejemplo: *Recuperar el nro de voluntario, nombre y apellido de los voluntarios que trabajan en la institución cuyo identificador es 60*.

```
SELECT nro_voluntario, nombre, apellido
FROM  voluntario
WHERE id_institucion= 60;
```

Resultado de la consulta

nro_voluntario	nombre	apellido
103	Alexander	Hunold
104	Bruce	Ernst
105	David	Austin
106	Valli	Pataballa
107	Diana	Lorentz

Condiciones de comparación

- Al utilizar los operadores de comparación el resultado puede ser:
  - Verdadero (T)
  - Falso (F)
  - Desconocido (U)
- Tener presente que si se comparan valores nulos usando los operadores de comparación el resultado será siempre FALSO porque un valor nulo no puede ser igual, mayor, distinto, etc. a otro valor.

```
SELECT nombre, apellido,
       e-mail
FROM  voluntario
WHERE id_tarea= 'ST_MAN';
```

```
SELECT nombre, apellido,
       e-mail
FROM  voluntario
WHERE id_tarea != 'ST_MAN';
```

Ejercicios

Muestre los apellidos, nombres, y e_mail de los voluntarios que llevan aportadas más de 1.000 horas, ordenados por apellido.

- ¿Cuál tabla consultar?
- ¿Cuáles atributos son necesarios y en qué cláusula?

http://dbases.exa.unicen.edu.ar/phppgadmin

Otros operadores de comparación

- Además de los operadores de comparación está disponible un operador especial [**NOT**] **BETWEEN** *:*
- **BETWEEN** trata a los valores de los extremos incluidos dentro del rango.
  - **BETWEEN** *x* **AND** *y* es equivalente a a >= x **AND** a <= y
  - **NOT BETWEEN** x **AND** y es equivalente a a < x **OR** a > y

Ejemplo: *Seleccionar las voluntarios cuyo número se encuentra entre 100 y 120*

```
SELECT nombre, apellido, e-mail
FROM voluntario
WHERE nro_voluntario BETWEEN 100 AND 120;
```

Operador LIKE

- No siempre se conoce el valor exacto a buscar. Se puede buscar coincidencias con un patrón de caracteres mediante el operador **LIKE**. También se emplea en la forma negativa **NOT LIKE**.
- Comodines %: cualquier secuencia de cero o más caracteres
  _ : denota un solo carácter

Ejemplo: Seleccionar los voluntarios cuya segunda letra del nombre sea a y luego tenga una n como carácter final.

```
SELECT nombre, apellido, e_mail
FROM voluntario
WHERE nombre LIKE '_a%n';
```

nombre	apellido	e_mail	telefono	fecha_nacimiento	id_tarea	nro_voluntario	horas_aportadas
Karen	Colmenares	KCOLMENA	515.127.4566	1999-08-10	PU_CLERK	119	2500.00
Jason	Mallin	JMALLIN	650.127.1934	1996-06-14	ST_CLERK	133	3300.00
Karen	Partners	KPARTNER	011.44.1344.467268	1997-01-05	SA_MAN	146	13500.00
Harrison	Bloom	HBLOOM	011.44.1343.829268	1998-03-23	SA_REP	169	10000.00

Operador IS [NOT] NULL

- Si una columna en particular carece de un valor se dice que contiene un **NULL**.
- **NULL** es un valor inaccesible, sin valor, desconocido o inaplicable.
- No representa ni un cero ni un espacio en blanco (el cero es un número y el espacio en blanco es un caracter). SOLO testean valores que son nulos.

Ejemplo: listar los voluntarios que no tengan coordinador.

```
SELECT nombre, apellido, e_mail
FROM voluntario
WHERE id_coordinador IS NULL;
```

nombre	apellido	e_mail	telefono	fecha_nacimiento	id_tarea	nro_voluntario	horas_aportadas	porcentaje
Steven	King	SKING	515.123.4567	1987-06-17	AD_PRES	100	24000.00	*NULL*

Operador IS [NOT] NULL

- Si se comparan valores nulos usando los otros operadores (=, >, etc.) el resultado será siempre FALSO porque un valor nulo no puede ser igual, mayor, distinto, etc. a otro valor.
- Si se desea incluir en el resultado los datos de aquellas columnas que tengan nulos hay que hacerlo explícitamente. Ejemplo listar los datos de los voluntarios sea menor o igual que 0,10.
  - Algunos porcentajes pueden ser nulos
  - Si deseo incluirlos en el resultado debo explicitar IS NULL.

```
SELECT * FROM voluntario
WHERE  porcentaje<=0.1
      OR  porcentaje IS NULL;
```

Condiciones de comparación compuestas

- Un operador lógico combina los resultados de dos condiciones para producir un único resultado basado en ellos, o invertir el resultado de una condición.
- Los operadores **AND** y **OR** se pueden usar para componer expresiones lógicas.
- El operador **AND** retorna VERDADERO si ambas condiciones evaluadas son VERDADERAS, mientras que el operador **OR** retorna VERDADERO si alguna de las condiciones es VERDADERA.
- El operador **NOT** invierte el resultado de la expresión.

Condiciones de comparación compuestas

Ejemplo: *Seleccionar los voluntarios que son coordinados por los voluntarios nro 100 o 124 y están trabajando para la institución cuyo código es 50.*

```
SELECT nro_voluntario, apellido,
       id_institucion, id_coordinador
FROM voluntario
WHERE (id_coordinador=100
   OR id_coordinador=124)
   AND id_institucion=50;
```

nro_voluntario	apellido	id_institucion	id_coordinador
120	Weiss	50	100
121	Fripp	50	100
122	Kaufling	50	100
123	Vollman	50	100
124	Mourgos	50	100
141	Rajs	50	124
142	Davies	50	124
143	Matos	50	124
144	Vargas	50	124
196	Walsh	50	124
197	Feeney	50	124
198	OConnell	50	124
199	Grant	50	124

Importante:

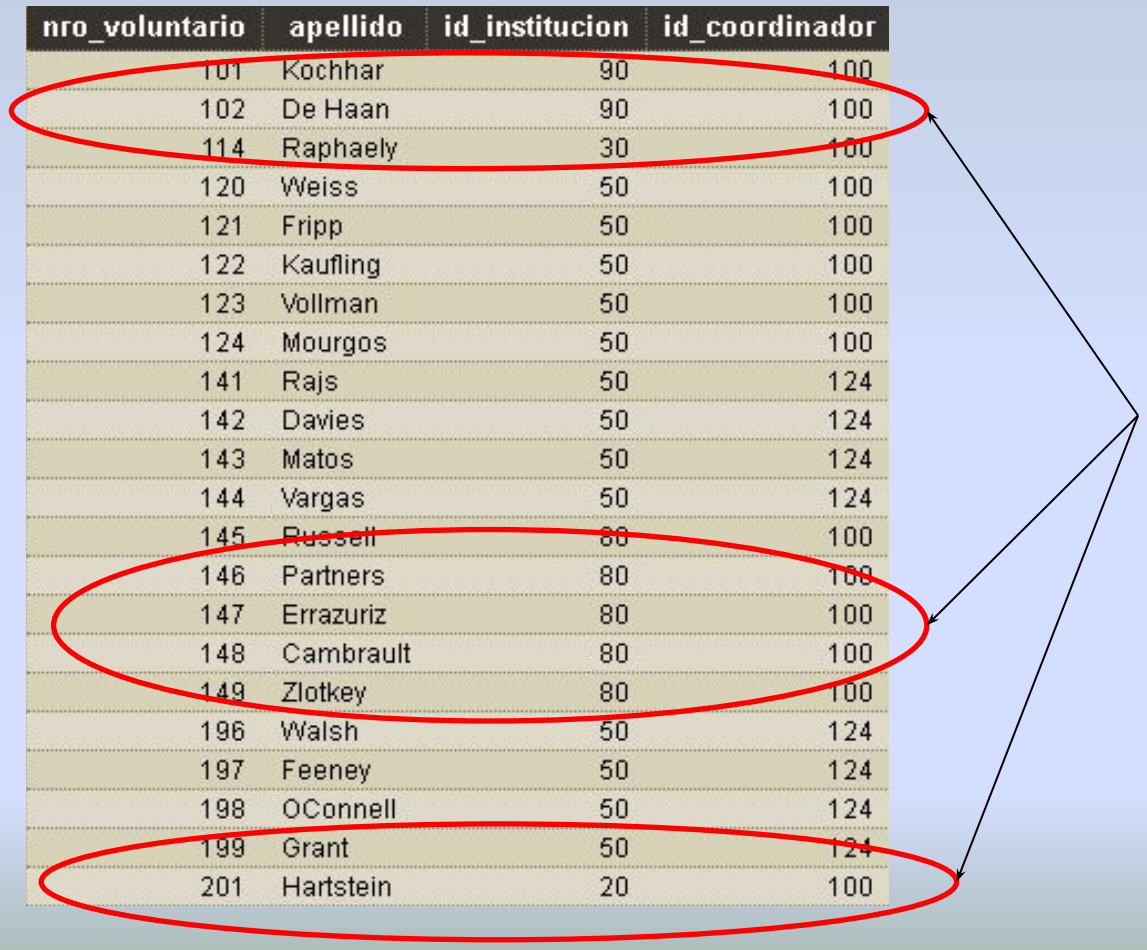
-Indentar las cláusulas

-Uso de paréntesis en las condiciones

Qué sucede si se eliminan ()?

...al eliminar ()

nro_voluntario	apellido	id_institucion	id_coordinador
101	Kochhar	90	100
102	De Haan	90	100
114	Raphaely	30	100
120	Weiss	50	100
121	Fripp	50	100
122	Kaufling	50	100
123	Vollman	50	100
124	Mourgos	50	100
141	Rajs	50	124
142	Davies	50	124
143	Matos	50	124
144	Vargas	50	124
145	Russell	80	100
146	Partners	80	100
147	Errazuriz	80	100
148	Cambrault	80	100
149	Zlotkey	80	100
196	Walsh	50	124
197	Feeney	50	124
198	OConnell	50	124
199	Grant	50	124
201	Hartstein	20	100



Estas filas no corresponden a la institución cuyo identificador es 50!!
No es válida la consulta sin los paréntesis en la condición!!!

Orden en la presentación de las tuplas

- El orden de las filas listadas en una consulta es indefinido, se puede utilizar la cláusula **ORDER BY** para ordenar las filas, y se debe colocar como última cláusula del **SELECT**.
- Por defecto si no se especifica el orden es ascendente (**ASC**), pero se puede especificar también **DESC** luego del nombre de la columna, para especificar un orden descendente.
- Se puede ordenar el resultado de una consulta por más de una columna (pueden ser todas las de la tabla).

Ejemplo *listar los apellidos ordenados descendentemente y nombres de los voluntarios que son coordinados por el voluntario 124.*

```
SELECT apellido, nombre
 FROM voluntario
WHERE  id_coordinador=124
ORDER BY apellido DESC, nombre;
```

apellido	nombre
Walsh	Alana
Vargas	Peter
Rajs	Trenna
OConnell	Donald
Matos	Randall
Grant	Douglas
Feeney	Kevin
Davies	Curtis

LIMIT and OFFSET (PostgreSQL)

- Permiten recuperar solamente un subconjunto de filas del total de la consulta.

```
SELECT lista de atributos
 FROM tabla/s
 [ORDER BY ... ]
 [LIMIT {numero | ALL}]
 [OFFSET numero];
```

- Debe ser usado siempre con la cláusula **ORDER BY**.
- La cláusula **LIMIT** limita la cantidad de filas a retornar.
- La cláusula **OFFSET** determina a partir de qué fila del resultado se retorna.

LIMIT and OFFSET (PostgreSQL)

Ejemplo: *seleccionar los datos de los voluntarios que corresponden a los 10 primeros voluntarios.*

```
SELECT apellido, nombre
FROM voluntario
ORDER BY nro_voluntario
LIMIT 10;
```

Ejemplo*: seleccionar los datos de los voluntarios a partir del 15° voluntario.*

```
SELECT apellido, nombre
FROM voluntario
ORDER BY nro_voluntario
LIMIT ALL
OFFSET 15;
```

Consultas SQL (cont)

- ✓ Sentencia del lenguaje SQL para la recuperación de datos
- ✓ Sintaxis básica:

```
SELECT * | [DISTINCT] {columna | expresion [alias],...}
FROM  lista_tablas
[WHERE condición_tuplas]
[ORDER BY lista_columnas [ASC|DESC]]
```

Funciones de agregación o agrupamiento

- ✓ Permiten resumir el resultado de una consulta, devolviendo un único valor agregado. Funciones básicas:

  MAX( ) → *valor máximo de la columna especificada*

  MIN ( ) → *valor mínimo de la columna especificada*

  SUM( ) → *suma de los valores (numéricos) de la columna*

  AVG( ) → *promedio de los valores (numéricos) de la columna*

  COUNT ( ) → *cantidad de tuplas seleccionadas*

- ✓ Se aplican a una columna, salvo COUNT que se puede indicar "*" (en el 1° caso no tiene en cuenta los nulos, en el 2° sí)
- ✓ Si se especifica DISTINCT, el cálculo se hace entre los valores distintos

```
SELECT  función de agrupamiento (columna), ...
FROM tabla/s
[ WHERE condición_tuplas ]
```

Funciones de agregación o agrupamiento

AVG, SUM y ***STDDEV*** sólo se usan para datos numéricos

```
SELECT  SUM(horas_aportadas) AS sum_horas,
        MAX(horas_aportadas) AS max_horas,
        MIN(horas_aportadas) AS min_horas,
        AVG(horas_aportadas) AS prom_horas
FROM voluntario;
```

MIN y **MAX** se pueden usar para cualquier tipo de dato

Ejemplo: *seleccionar la fecha de nacimiento del voluntario más joven y del de más edad*

```
SELECT  MAX(fecha_nacimiento) AS fnac_volunt_mas_joven,
        MIN (fecha_nacimiento) AS fnac_volunt_mas_viejo
FROM voluntario;
```

Funciones de agregación o agrupamiento

- **COUNT(*)** devuelve el número de filas de una tabla

```
SELECT   COUNT(*)
  FROM  voluntario;
```

- **COUNT(**columna**)** devuelve el número de filas con valores no nulos para la columna

Ejemplo: *cantidad de instituciones en las que trabajan los voluntarios*

```
SELECT   COUNT(id_institucion)
  FROM  voluntario;
```

Ejemplo: *cantidad de instituciones diferentes en que trabajan los voluntarios*

```
SELECT   COUNT(distinct id_institucion)
  FROM  voluntario;
```

Funciones de agrupamiento y valores nulos

- Las funciones de grupo ignoran los valores nulos del atributo.

```
SELECT   AVG(porcentaje) AS
     Porcentaje_promedio
FROM voluntario;
```

porcentaje_promedio
0.22285714285714285714

- La función **COALESCE**(columna, valor_reemplazo) fuerza a las funciones de grupo a que incluyan valores nulos, retornando un valor en ocurrencia de un nulo.

```
SELECT   AVG(COALESCE(porcentaje, 0)) AS
     Porcentaje_promedio
FROM voluntario;
```

porcentaje_promedio
0.07289719626168224299

Ejemplos de Funciones de Grupo o Agrupamiento

```
SELECT   COUNT(*) AS
    cantidad_de_voluntarios
FROM voluntario;
```

cantidad_de_voluntarios
107

Alias de columna

```
SELECT   SUM(horas_aportadas) AS
    Horas_trabajadas
FROM voluntario v WHERE
    v.id_coordinador=120;
```

horas_trabajadas
22100.00

Alias de tabla

```
SELECT   MAX(horas_aportadas) maximo,
    MIN(horas_aportadas) minimo,
    MAX(horas_aportadas) –
    MIN(horas_aportadas) diferencia
FROM voluntario v
  WHERE v.id_coordinador=120;
```

maximo	minimo	diferencia
3200.00	2200.00	1000.00

Selección sobre grupos de datos

- La cláusula **GROUP BY** en una sentencia SELECT forma grupos con las filas que tienen valores en común para la lista de columnas especificadas para el grupo (*puede ser una o más columnas*)

```
SELECT  lista_columnas [, función de agrupamiento (columna)]
FROM  lista_tablas
[ WHERE  condición_tuplas ]
GROUP BY  lista_columnas;
```

- Se suele usar con **funciones de agrupamiento** para obtener datos resumidos por cada grupo
- *Retorna una única fila por cada grupo*

Selección sobre grupos de datos

Ejemplo: *¿cuántos voluntarios trabajan en cada institución?*

nro_voluntario	apellido	id_institucion
121	Fripp	50
196	Walsh	50
147	Errazuriz	80
103	Hunold	60
115	Khoo	30
185	Bull	50
158	McEwen	80
175	Hutton	80
167	Banda	80
187	Cabrio	50
193	Everett	50
104	Ernst	60
179	Johnson	80
153	Olsen	80
162	Vishney	80
142	Davies	50
109	Faviet	100
163	Greene	80
105	Austin	60
165	Lee	80

...

```
SELECT   id_institucion, COUNT(*) AS
    cantidad_voluntarios
 FROM voluntario
 GROUP BY id_institucion;
```

id_institucion	cantidad_voluntarios
90	3
NULL	1
20	2
100	6
40	1
110	2
80	34
70	1
50	45
60	5
30	6
10	1

Selección sobre grupos de datos

- **Importante**: Todas las columnas en el SELECT, excepto funciones de grupo, deben estar en la cláusula GROUP BY

```
SELECT  id_institucion, id_tarea, AVG(porcentaje)
  FROM voluntario
  GROUP BY id_institucion;
```

```
[42803] ERROR: column "voluntario.id_tarea" must appear in the GROUP BY
clause or be used in an aggregate function
```

- Las columnas en la cláusula **GROUP BY** pueden no estar en la lista del **SELECT**

```
SELECT  AVG(porcentaje)
  FROM voluntario
  GROUP BY id_institucion;
```

Restringir los resultados de los grupos

Se puede anexar la cláusula **HAVING** para restringir grupos

1. Las filas se agrupan por la/s columnas especificada/s en el GROUP BY
2. Se aplica la función de agrupamiento
3. Se muestran los grupos que satisfacen la condición en el **HAVING** (normalmente involucran funciones de grupo)

```
SELECT  lista_columnas, función de agrupamiento (columna)
FROM  lista_tablas
[ WHERE  condición_tuplas ]
GROUP BY  lista_columnas
[ HAVING  condición_grupo ];
```

Ejemplo: *listar los coordinadores con más de 7 voluntarios*

```
SELECT   id_coordinador, COUNT(*)
  FROM voluntario
  GROUP BY id_coordinador
  HAVING COUNT(*) > 7;
```

Restringir los resultados de los grupos

- ✓ No utilizar la cláusula **HAVING** para aplicar condiciones a las tuplas → se debe utilizar la cláusula **WHERE**
- ✓ No utilizar la cláusula **WHERE** para restringir grupos → se debe utilizar la cláusula **HAVING**
- ✓ No se pueden utilizar funciones de grupo en la cláusula **WHERE**

```
SELECT   id_coordinador, COUNT(*)
  FROM voluntario
  WHERE COUNT(*) >7
  GROUP BY id_coordinador;
```

Error de SQL:

```
ERROR:  aggregates not allowed in WHERE clause at character 105
```

En la declaración:

```
SELECT id_coordinador, count(*) as Cantidad_voluntarios

 FROM  unc_esq_voluntario.voluntario  where  count(*) >7
group by id_coordinador

;
```



WHERE: para las restricciones a nivel de filas

HAVING: para las restricciones a realizar sobre grupos

Importante: ejecución de consultas

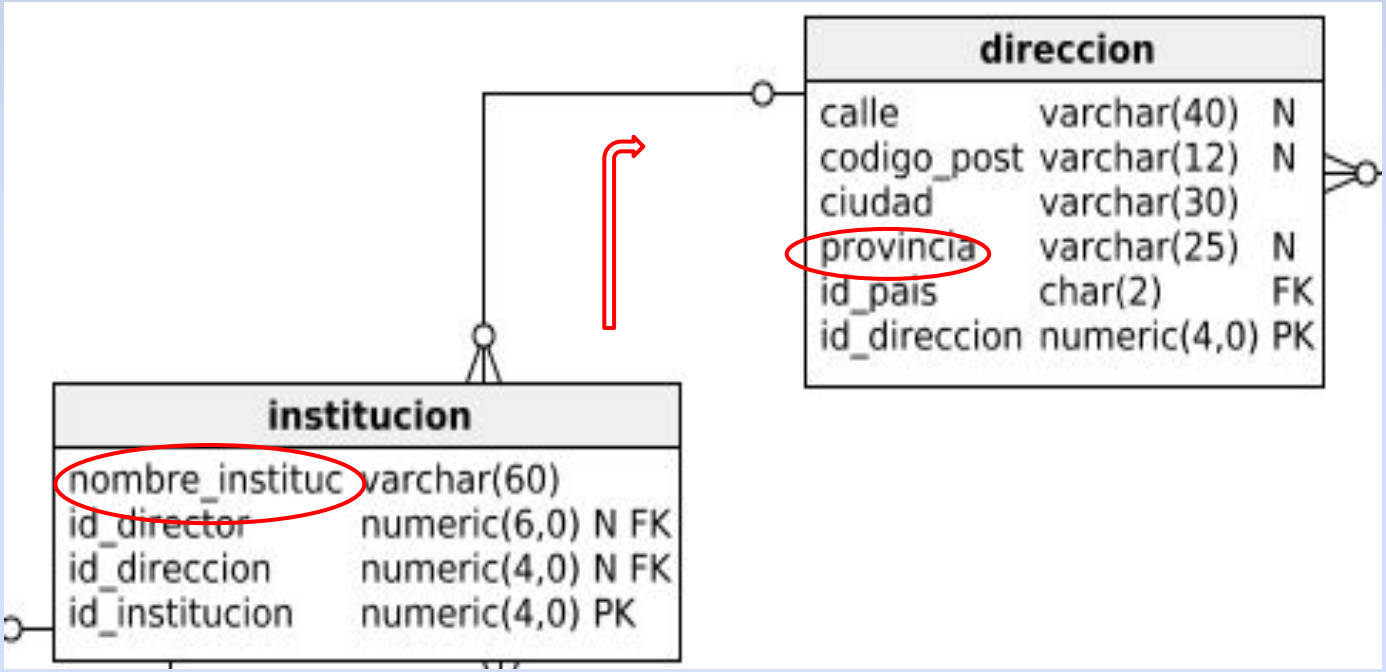
Orden de Ejecución de un Query en **TODAS** las Bases de Datos Relacionales:

1. **FROM** - Se obtiene el DataSet (Conjunto de Datos) sobre el cual se operará
2. ***WHERE** - Se filtra el DataSet del punto 1
   - Se recorre fila por fila y a cada una se le aplica la condición lógica especificada
3. ***GROUP BY** - Se parte el DataSet resultante del punto 2
   - Se hace en grupos de acuerdo a las columnas especificadas en el GROUP BY
4. ****HAVING** - Se filtran los GRUPOS (No el "contenido" de los mismos) del punto 3 de acuerdo a la condición lógica especificada
5. **SELECT** - el DataSet obtenido se proyecta
   - Se vuelve a recorrer fila por fila del Dataset obtenido en el punto 4 y se proyecta por lo indicado en el SELECT
6. ***ORDER BY** - Se ordena el DataSet obtenido en el punto 5
7. ***LIMIT/OFFSET** Se limita el resultado ya obtenido en el punto 5
8. Se envía el **Dataset final** al cliente del **DBMS**

** Se ejecuta ese paso si es que existe la cláusula en la consulta*
*** Se puede usar si sólo si existe el GROUP BY, NO SE PUEDE USAR SIN EL MISMO*

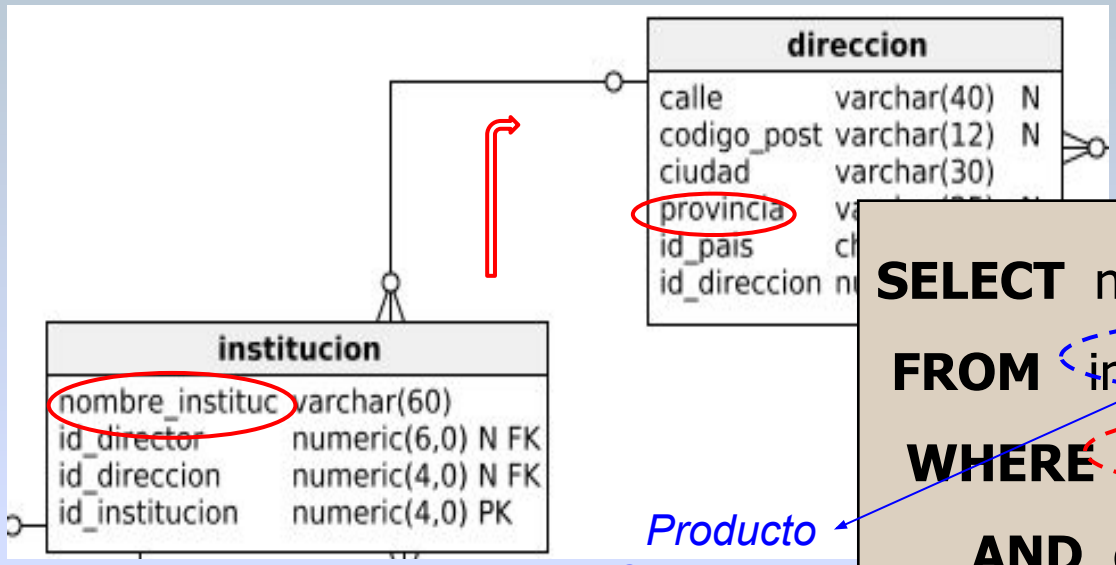
Consultas de más de una tabla

Si se requiere, por ejemplo: seleccionar el nombre de las instituciones del estado (provincia) de Texas



Se puede resolver de distintas maneras…

Consultas de más de una tabla



Una posibilidad:

```
SELECT  nombre_institucion
FROM    institucion i, direccion d
WHERE   i.id_direccion = d.id_direccion
AND     d.provincia = 'Texas' ;
```

Producto Cartesiano

Condición de ensamble entre tablas

El **producto cartesiano** (o "**cross join**") combina cada tupla de la primera tabla con cada tupla de la segunda tabla, para obtener tuplas compuestas por atributos de las dos relaciones combinadas

- ✓ genera todas las posibles combinaciones de filas entre las dos tablas
- ✓ luego filtra las tuplas que satisfacen la condición

Ensambles Internos

- El operador **[INNER] JOIN** combina las tuplas de una tabla con las tuplas de otra tabla que satisfacen una condición especificada (mediante **ON**) para obtener tuplas con los atributos de ambas relaciones

  Se resuelve más eficientemente que el producto cartesiano

```
SELECT *
 FROM institucion i INNER JOIN direccion d
   ON i.id_direccion = d.id_direccion;
```

se puede omitir

- Se antepone el nombre de cada tabla (o su alias) para evitar ambigüedades cuando hay nombres de columnas que se repiten en el resultado y la concatenación no se hace a través de ellas
- Si la/s columna/s por la/s cual/es se ensambla se llama/n igual en ambas tablas → se puede indicar USING

```
SELECT * FROM institucion JOIN direccion
 USING (id_direccion);
```

Ensambles Internos

- **NATURAL JOIN:** caso especial de join en el que las tablas se combinan en función de las columnas con igual nombre en ambas tablas (las columnas por las que se hace el ensamble aparecen una sola vez en el resultado)

```
SELECT *
 FROM institucion NATURAL JOIN direccion ;
```

- Se debe tener precaución al utilizarlo ya que se ensambla por todas las columnas cuyos nombres coincidan
  → podrían generarse errores inadvertidos (particularmente ante la creación de nuevas columnas en alguna de las tablas involucradas)

Ejemplo: ¿qué resultaría si se agrega una columna e-mail a institucion?

```
SELECT *
 FROM voluntario NATURAL JOIN institucion;
```

Ensambles Internos

- El INNER JOIN sólo se queda con las filas que tienen valores idénticos en las columnas de las tablas que combina
- Pero… cuando esas columnas aceptan nulos → NO aparecerán en el resultado aquellas tuplas que poseen nulos en esas columnas

Ejemplo: listar id y nombre de las instituciones y el nombre del director

```
SELECT i.id_institucion, nombre_institucion, nombre as nom_director
  FROM  institucion i  JOIN  voluntario v
    ON  i.id_director = v.nro_voluntario ;
```

si hay instituciones que no tienen director (id_director es nulo) → no aparecen en el resultado (ya que no tienen ningún voluntario con quien combinarse)

Ensambles Externos

- **OUTER JOIN**: permite recuperar tuplas de una o ambas tablas, incluso si no hay coincidencias en la condición de ensamble (debido a la presencia de nulos) y completa con NULL los valores inexistentes
  - LEFT [OUTER] JOIN: *se incluyen <u>todas</u> las tuplas de la tabla a la izquierda del JOIN, completando con nulos las columnas de la otra tabla (derecha) que no tienen correspondencia*
  - RIGHT [OUTER] JOIN: *se incluyen <u>todas</u> las tuplas de la tabla de la derecha, completando con nulos las columnas de la tabla izquierda sin correspondencia*
  - FULL [OUTER] JOIN: es la combinación de los anteriores *(se incluyen todas las tuplas de ambas tablas, completando con nulos las columnas correspondientes de la otra tabla en cada caso)*

Su formato es:

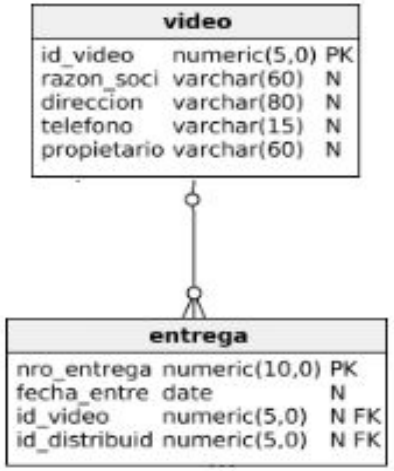
```
SELECT nombre_columnas_a_seleccionar
FROM t1 [NATURAL] [LEFT|RIGHT|FULL] [OUTER] JOIN t2
{ON condiciones| [USING (columna [,columna...])}
[WHERE condiciones];
```

Ensambles Externos

Ensamble externo: Operador RIGHT JOIN

Listar las entregas de peliculas y todos los videos

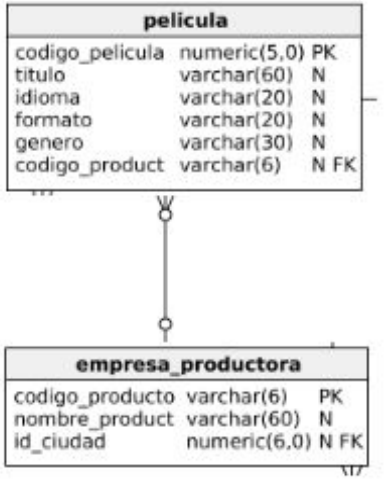
```
SELECT *
FROM entrega RIGHT JOIN video
  using (id_video) ;
```



Ensamble externo: Operador LEFT JOIN

Listar todas las empresas productoras junto con las películas que producen:

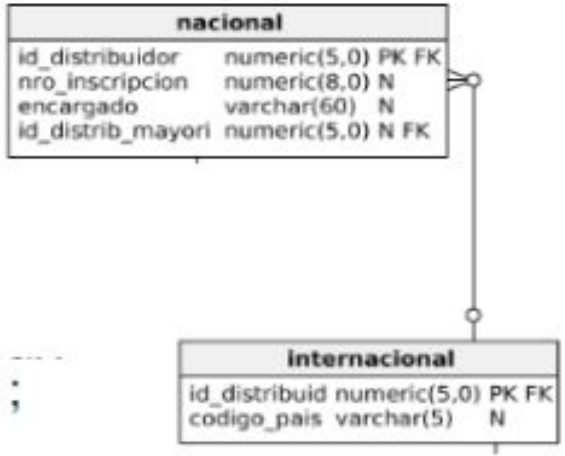
```
SELECT *
FROM empresa_productora LEFT JOIN pelicula;
  using (codigo_productora) ;
```



Ensambles Externos

Ensamble externo: Operador FULL JOIN

Listar todos los distribuidores nacionales e internacionales



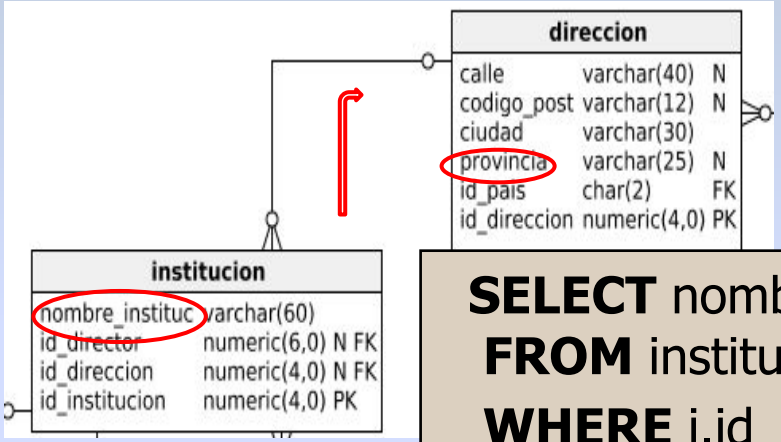

```
SELECT *
FROM nacional N FULL JOIN internacional I
    ON (N.id_distrib_mayorista = I.id_distribuidor) ;
```

→ Lista todos los distribuidores nacionales tengan o no distribuidor mayorista y tambien se incluiran los distribuidores internacionales que no estaban asociados a ningun distribuidor nacional.

Nota: la cláusula ON la debo especificar cuando las columnas de ensamble (join) no coinciden en nombre

Consultas anidadas

- ✓ La cláusula WHERE admite SELECTs anidados (o subconsultas)
- ✓ Ejemplo: listar el nombre de las instituciones del estado (provincia) Texas




```
SELECT nombre_institucion
 FROM institucion i
 WHERE i.id_direccion = (SELECT id_direccion
                         FROM direccion d
                         WHERE d.provincia = 'Texas');
```

?

Importante: ¿qué ocurre si hay más de un resultado en la subconsulta?

- Usar operadores de fila única o **single-rows (=, >, <, <>, >=, <=)** para subconsultas que retornan una única fila
- Usar operadores de múltiples filas o **multiple-rows (IN, ANY, ALL)** para subconsultas que pueden retornar varias filas

Consultas con subconsultas de una fila

```
SELECT nombre, apellido
 FROM   voluntario
WHERE horas_aportadas = (SELECT MIN(horas_aportadas)
                         FROM voluntario);
```

```
SELECT id_tarea, MIN(horas_aportadas)
   FROM voluntario
GROUP BY id_tarea
HAVING MIN(horas_aportadas) > (SELECT AVG(horas_aportadas)
                               FROM voluntario);
```

Es responsabilidad de quien escribe el SELECT asegurar que la subconsulta devolverá una sola fila. Si el subquery devuelve más de una fila, dará error de ejecución

Consultas con subconsultas de varias filas

OPERADOR	SIGNIFICADO
IN	Retorna TRUE si el valor está incluido en los valores retornados por la subconsulta
EXISTS	Retorna TRUE si el subquery devuelve al menos una fila. FALSE si devuelve 0 filas
ANY	Retorna TRUE si la comparación es TRUE para al menos un valor retornado por el subquery
ALL	Retorna TRUE si la comparación es TRUE para todos los valores retornados por el subquery

Consultas con subconsultas de varias filas

Operador [NOT] IN : Permite determinar si los valores de una columna, o conjunto de ellas, están contenidos (o no) en una lista definida o dentro de otra tabla (subconsulta)

Ejemplos: Seleccionar nro, nombre y apellido de los voluntarios que realizan tareas cuyo id es SA_MAN o ST_MAN

```
SELECT nro_voluntario, nombre, apellido
FROM voluntario
WHERE id_tarea IN ( 'SA_MAN', 'ST_MAN' );
```

Seleccionar el nro, nombre y apellido de los voluntarios que realizan tareas cuyo nombre comienza con ORG

```
SELECT nro_voluntario, nombre, apellido
FROM voluntario
WHERE id_tarea IN
        ( SELECT id_tarea
          FROM tarea
          WHERE nombre_tarea LIKE 'ORG%');
```

De manera similar se pueden seleccionar los voluntarios que NO realizan dichas tareas usando **NOT IN**

Consultas con subconsultas de varias filas

Operador EXISTS: Se refiere únicamente a la determinación de si la subconsulta devuelve alguna fila

Si devuelve una o más filas, el predicado se evalúa como verdadero; de lo contrario, se evalúa como falso

Tiene la forma de una **consulta correlacionada**:

```
SELECT lista de atributos
FROM tabla_externa E
WHERE EXISTS
        (SELECT 1
         FROM tabla_interna I
         WHERE E.atributo1 = I.atributo1
           …..
             AND E.atributoN = I.atributoN);
```

Operador EXISTS

El operador EXISTS comprueba la existencia de filas en el conjunto de filas del resultado de la subconsulta.

- Si se encuentra un valor de fila de la subconsulta entonces:
  - Se coloca la condición como verdadera
  - La búsqueda no continúa en la consulta interna
- Si no se encuentra un valor de fila en la subconsulta entonces:
  - Se coloca la condición como falsa
  - la búsqueda continúa en la consulta interna para el siguiente valor de la tabla externa.

Ejemplo: Seleccionar el nro., nombre y apellido de los voluntarios que realizan tareas cuyo código comienza con ORG

```
SELECT nro_voluntario, nombre, apellido
FROM voluntario V
WHERE  EXISTS ( SELECT  1  FROM tarea T
                WHERE V.id_tarea = T.id_tarea
                AND nombre_tarea LIKE 'ORG%' );
```

Operador NOT EXISTS

Para el NOT EXISTS se comprueba que no existan filas en el conjunto de filas del resultado de la subconsulta interna

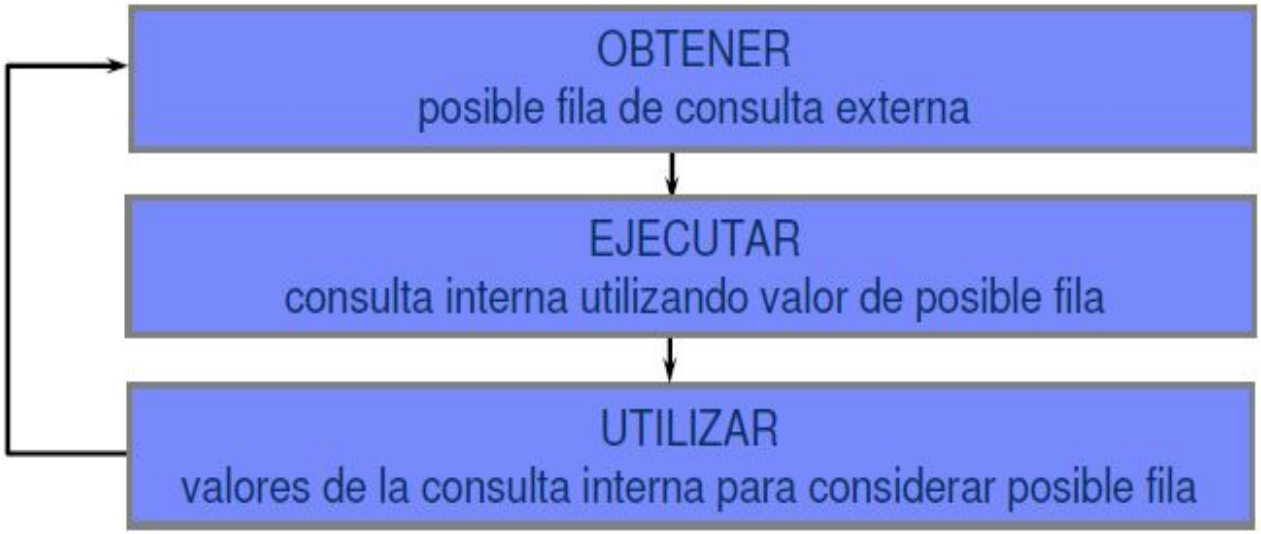
Ejemplo: Seleccionar el nro, nombre y apellido de los voluntarios que NO realizan tareas cuyo nombre comienza con ORG

```
SELECT nro_voluntario, nombre, apellido
FROM voluntario V
WHERE NOT EXISTS  (SELECT 1
                   FROM esq_vol_tarea T
                   WHERE V.id_tarea = T.id_tarea
                   AND nombre_tarea LIKE 'ORG%');
```

- Las consultas con IN y EXISTS deberían dar los mismos resultados si la consulta está bien construida, aunque la ejecución sean diferente
- Importante: NOT IN y NOT EXISTS no son equivalentes:
  - no todo lo que se resuelve con NOT EXISTS se puede plantear con NOT IN
  - considerar el comportamiento en caso de presencia de nulos

Subconsultas Correlacionadas

Se utilizan para el procesamiento fila a fila. Cada subconsulta se ejecuta una vez para cada fila de la consulta externa.




```
SELECT columna1, columna2, ...
FROM   tabla1  externa
WHERE  columna1 <operador>
                    (SELECT  columa1, columna2
                     FROM    tabla2 intera
                     WHERE   expr1 =
                               externa.expr2);
```

Uso del operador EXISTS

Seleccionar el nro., nombre y apellido de los voluntarios que son [que no son] coordinadores

```
SELECT nro_voluntario, nombre, apellido
FROM voluntario V1
WHERE [NOT] EXISTS
        (SELECT 1
        FROM voluntario V2
        WHERE V1.nro_voluntario = V2.id_coordinador);
```

Tratamiento de los nulos

Pueden surgir complicaciones cuando los valores nulos participan en operaciones aritméticas o de comparación (deberían evitarse siempre que sea posible)

Un nulo indica *valor desconocido o no existente*

- cualquier operación aritmética (+, –, * , /) que los incluya devuelve nulo (Ejemplo: 100 + *null* devuelve *null*)

- cualquier comparación (como <, <=, >, >= y !=) que los incluya no se puede decir si el resultado es verdadero o falso → se evalúa como *desconocido*

Lógica trivaluada

Las comparaciones que incluyen nulos pueden aparecer dentro de expresiones booleanas que incluyan operaciones AND, OR, NOT → se debe advertir la forma en que estas operaciones tratan el valor lógico desconocido

LÓGICA DE TRES VALORES :

VERDADERO (True)
FALSO (False)
DESCONOCIDO (Unknown)

& (AND)	T	U	F
T	T	U	F
U	U	U	F
F	F	F	F

I (OR)	T	U	F
T	T	T	T
U	T	U	U
F	T	U	F

~(NOT)	
T	F
U	U
F	T

Lógica trivaluada y SQL

- En las funciones de agregación SUM(), AVG(), MIN(), MAX(),... se eliminan implícitamente los nulos del cálculo (en particular, si cada ítem de datos en una columna es null, SUM(), AVG(), MIN(), MAX() devuelven null)
- COUNT(…) no cuenta las filas en las cuales el atributo entre () es null, a menos que éste sea `*´ ya que COUNT(*) devuelve el número total de filas sin depender de los valores de una columna
- Para ORDER BY, null es el 'mayor'
- Para DISTINCT, null es duplicado de null

Lógica trivaluada y SQL

La condición del operador IN (o NOT IN) puede plantearse indicando la lista de valores dada por extensión, entonces:

x **IN** (v1, v2, …., vn) es equivalente a

((x=v1) or (x=v2) … or (x=vn))

→ evalúa a desconocido para null, verdadero para cualquier valor v1...vn y falso para el resto

x **NOT IN** (v1, v2, …., vn) es equivalente a

((x<>v1) and (x<>v2) … and (x<>vn))

→ evalúa a desconocido para null, verdadero para cualquier valor distinto de v1...vn y falso para el resto

¿Qué problema puede haber ?

¿Valores por defecto vs NULLs?

Usar valores por defecto como alternativa al uso de NULLs:

- ✓ Si un valor es desconocido, se rellena con un valor en particular
- ✓ Son valores, por lo tanto no es necesario aplicar la Lógica Trivaluada
- ✓ Suelen tener más significado que los NULLs, por ejemplo: 'ninguno', 'desconocido', 'no aplicable' para nombres; -1 para precio, cantidad, etc.
- ✓ Pero… al ser valores 'reales' (no nulos):
  - pueden actualizarse como cualquier otro valor *(¿qué pasaría si se usa -1 para precio y accidentalmente se hace UPDATE Producto SET precio= precio + 5 ?)*
  - se necesita encontrar un valor que no pueda utilizarse en ningún caso (distinguible) - se corre el riesgo de interpretación incorrecta
  - en SQL los valores por defecto deben ser del mismo tipo que los valores de la columna *(No usar 'desconocido' en una columna de enteros!)*

¿Entonces? Los valores por defecto NO deberían usarse cuando pueden 'confundirse' con valores reales; en ese caso convendría usar NULLs.

Siempre tener presente donde se pueden encontrar nulos! → revisar en el planteo de sentencias las sentencias SELECT



Álgebra Relacional

- Es un lenguaje formal con una serie de operadores que trabajan sobre una o varias relaciones para obtener otra relación como resultado
- Permite especificar consultas en BD relacionales y se utiliza como representación para traducir y optimizar consultas

Operadores tradicionales sobre conjuntos:

Producto Cartesiano, Unión, Diferencia, Intersección

$\times$ $\cup$ $-$ $\cap$

Operadores específicos para BD relacionales:

Selección, Proyección, Ensamble (Join), División

σ π $\bowtie$ $\div$

Operadores de Conjuntos

Implementación de los operadores del álgebra en SQL:

PRODUCTO CARTESIANO: se indica en la cláusula FROM las tablas involucradas, separadas por ","

Para la unión, intersección y diferencia:

expresión_SELECT

UNION | INTERSECT | EXCEPT [ALL]

expresión_SELECT

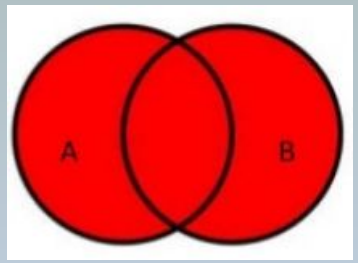
Importante:

- Las operaciones del **álgebra relacional** generan relaciones *sin elementos repetidos*, se debe especificar **ALL** si se quieren preservar los repetidos en el resultado
- A y B deben ser ***unión-compatibles*** (mismo esquema y dominio de definición)

Álgebra Relacional vs. Álgebra de Bolsas

- Las operaciones del **álgebra relacional** generan relaciones *sin elementos repetidos*
- Pero… **SQL permite tuplas repetidas** en el resultado de consultas (a menos que se indique explícitamente removerlas)
- Los motores de BD relacionales en realidad operan con un **álgebra de bolsas** (admiten repeticiones de tuplas)
  - *La eliminación de duplicados es una operación costosa*
  - *Es posible que se requiera conocer todos los resultados de una consulta, incluso si están repetidos*
- Las operaciones del álgebra relacional se redefinen para el álgebra de bolsas

Unión: UNION [ALL]



- Devuelve las tuplas que están en una o en ambas expresiones SELECT que se están uniendo (deben ser unión-compatibles)
- Operación conmutativa: A ∪ B = B ∪ A
- si se usa ALL: si un resultado aparece *m* veces en A y *n* veces en B, en el resultado aparecerá *m + n* veces

Ejemplo: Seleccionar las tareas desarrolladas tanto en la actualidad como históricamente

```
SELECT id_tarea
FROM voluntario
  UNION
SELECT id_tarea
FROM historico;
```

id_tarea
AD_VP
ST_MAN
HR_REP
AC_MGR
PR_REP
PU_MAN
FI_MGR
ST_CLERK
AD_PRES
PU_CLERK
MK_MAN
AD_ASST
FI_ACCOUNT
IT_PROG
SA_REP
MK_REP
SA_MAN
SH_CLERK
AC_ACCOUNT

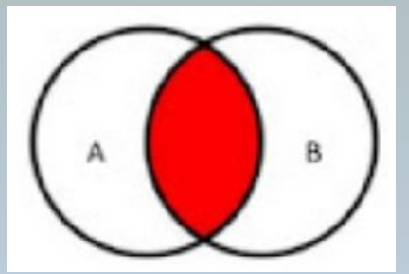
19 fila(s)

```
SELECT id_tarea
FROM voluntario
  UNION ALL
SELECT id_tarea
FROM historico;
```

id_tarea
IT_PROG
AC_ACCOUNT
AC_MGR
MK_REP
ST_CLERK
ST_CLERK
ST_CLERK
ST_CLERK
ST_CLERK
ST_CLERK
MK_MAN
MK_REP
HR_REP
PR_REP
AC_MGR
AC_ACCOUNT

118 fila(s)

Intersección: INTERSECT [ALL]



- Devuelve las tuplas que están en ambas expresiones SELECT que se intersecan (deben ser unión-compatibles)
- Operación conmutativa: A ∩ B = B ∩ A
- si se usa ALL: si un resultado aparece *m* veces en A y *n* veces en B, en el resultado aparecerá min(*m*, *n*) veces.

Ejemplo: Listar las tareas desarrolladas actualmente y que también se realizaron anteriormente

```
SELECT id_tarea
FROM voluntario
 INTERSECT
SELECT id_tarea
FROM historico;
```

id_tarea
SA_REP
AC_MGR
AD_ASST
SA_MAN
MK_REP
IT_PROG
ST_CLERK
AC_ACCOUNT

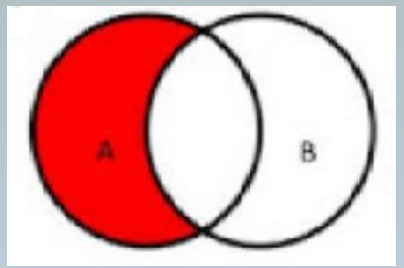
8 fila(s)

```
SELECT id_tarea
FROM voluntario
  INTERSECT ALL
SELECT id_tarea
FROM historico;
```

id_tarea
SA_REP
AC_MGR
AD_ASST
SA_MAN
SA_MAN
MK_REP
IT_PROG
ST_CLERK
ST_CLERK
AC_ACCOUNT

10 fila(s)

Diferencia: EXCEPT [ALL]



- Devuelve las tuplas resultantes de la primera expresión SELECT que no están en la segunda expresión (deben ser unión-compatibles)
- Operación NO conmutativa: A – B ≠ B – A
- si se usa ALL: si un mismo resultado aparece *m* veces en A y *n* veces en B, en el resultado aparecerá max(*m* – *n*, 0) veces

Ejemplo: Listar las tareas desarrolladas actualmente y que no se realizaron antes

```
SELECT id_tarea
FROM voluntario
 EXCEPT
SELECT id_tarea
FROM historico;
```

id_tarea
SH_CLERK
AD_VP
PU_MAN
FI_MGR
PU_CLERK
HR_REP
ST_MAN
MK_MAN
PR_REP
AD_PRES
FI_ACCOUNT

11 fila(s)

```
SELECT id_tarea
FROM voluntario
 EXCEPT ALL
SELECT id_tarea
FROM historico;
```

id_tarea
SH_CLERK
SH_CLERK
SH_CLERK
SH_CLERK
SA_REP
PR_REP
AD_PRES
FI_ACCOUNT
FI_ACCOUNT
FI_ACCOUNT
FI_ACCOUNT
FI_ACCOUNT

97 fila(s)

Operadores para BD Relacionales

Implementación de los operadores en SQL:

SELECCIÓN: se utiliza SELECT para recuperar datos de una o más tablas según criterios (condiciones) de selección

PROYECCIÓN: se especifica en la sentencia SELECT las columnas que deseas recuperar en la consulta

ENSAMBLE: el operador JOIN se utiliza para combinar dos o más tablas en función de una condición de coincidencia entre las filas de esas tabla (INNER, LEFT, RIGHT y FULL JOIN)

DIVISIÓN: se usa para resolver, dado un conjunto de valores A y un conjunto de valores B, ¿qué valores en A se relacionan con todos los valores en B? (no hay un operador en SQL que lo implemente, hay que "simularlo")

División

- Si A –*divisor* – es una relación con esquema (A1, …., An, B1,…., Bm)
- y B –*dividendo* – es una relación con esquema (B1,…., Bm),
- da como resultado otra relación con esquema (A1, …., An) que contiene las tuplas de A que están acompañadas por **TODAS** las ocurrencias de B
- Operación NO conmutativa: $A \div B \neq B \div A$

Ejemplo: Listar los alumnos inscriptos en todas las materias:

$$(ALUMNO \bowtie_C INSCRIPCION) \div \pi_{Id_m} (MATERIA)$$

C: condición del JOIN por igualdad de id_alum

ALUMNO

Id_alum	Nom_alum
1	Martin
2	Juan
3	Laura

INSCRIPCION

Id_alum	Id_m
1	M1
1	M2
2	M2

MATERIA

Id_m	Nom_mat
M1	Materia 1
M2	Materia 2

División

Ejemplo: Listar los alumnos inscriptos en todas las materias

ALUMNO

Id_alum	Nom_alum
1	Martin
2	Juan
3	Laura

INSCRIPCION

Id_alum	Id_m
1	C1
1	C2
2	C2

MATERIA

Id_m	Nom_mat
M1	Materia 1
M2	Materia 2

```
SELECT *
FROM alumno A
WHERE NOT EXISTS
        ( SELECT M.Id_m
          FROM materia M
             EXCEPT
          SELECT I.Id_m
          FROM inscripcion I
          WHERE I.Id_alum=A.Id_alum );
```

```
SELECT *  FROM alumno A
WHERE NOT EXISTS
        ( SELECT M.Id_m
          FROM materia M
          WHERE NOT EXISTS
                ( SELECT I.Id_m
                  FROM inscripcion I
                  WHERE I.Id_alum=A.Id_alum
                      AND I.id_m=M.Id_m) );
```

Otra posibilidad: comparar si la cantidad de materias en las que se inscribió cada alumno (en Inscripciones) coincide con la cantidad de materias que hay (en Materias)



Para recordar

- La sentencia SQL empleada para la recuperación de los datos a partir de las tablas cargadas en la base de datos es el **SELECT**
- **SELECT** identifica las columnas a recuperar – **EL QUÉ**
- **FROM** identifica la/s tabla/s - **DE DÓNDE** obtener los datos
- **WHERE** se usa para realizar las restricciones sobre los datos
- Para eliminar los valores repetidos se debe usar la cláusula **DISTINCT**
- Los operadores de comparación se utilizan en la cláusula WHERE para comparar expresiones
- **ORDER BY** puede usarse para ordenar las filas, y se debe colocar como última cláusula de la sentencia SELECT
- Las funciones de agregación operan sobre conjuntos de filas para proporcionar un resultado por grupo
- **GROUP BY** *especifica cómo se deben agrupar las filas seleccionadas. Todas las columnas de la lista SELECT, excepto las funciones de grupo, deben estar en la cláusula GROUP BY.* **No** se pueden utilizar funciones de agrupamiento en la cláusula WHERE, para esto se usa **HAVING**
- *Usar paréntesis y sangrías para mejorar la legibilidad*

Recomendaciones

- ✓ Completar con textos de la bibliografía recomendada (en todos se explica SQL y se plantean ejemplos)
- ✓ Consultar la documentación del SQL estándar y de PostgreSQL
- ✓ Resolver ejercicios propuestos en la práctica
- ✓ Probar ejercicios en PostgreSQL (accesible vía web, ver URL y datos para conexión), aprovechar las clases prácticas para consultar!



Practicar…

Practicar…

Practicar… !!!

Bibliografía

- Elmasri R., Navathe S., "Fundamentals of Database Systems", Pearson, 7° ed., 2016 (o anteriores)
- Ramakrishnan R., Gehrke J. "Database Management Systems", 3° ed., McGraw-Hill, 2002
- Silberschatz, A., Korth, H, Sudarshan, S, "Database System Concepts", 6° ed., WCB/McGraw Hill, 2010
- SGBD PosgreSQL - documentación: www.postgresql.org/docs/

